

TECHNICAL REPORT

Machine Learning Algorithms on MNIST

Phase 2 report

May 2026

Ahmed Walid Elsayed Zahran Ali Mostafa 23P0038

Seif Tamer Safwar Elsherif 23P0240

Abdelrahman Khaled Gaber Ibrahim 23P0065

Asaad Ramzy Asaad 23P0228

Presented to:

Dr. Hossam Abdelmunim

Eng. Ali Ahmed

Table of Contents

Abstract.....	4
1. Introduction.....	4
2. Dataset and Preprocessing.....	5
2.1 MNIST Dataset.....	5
2.2 Preprocessing Pipeline.....	5
Step 1 — Pixel Normalisation	5
Step 2— PCA (Softmax Regression Only).....	5
3. K-Nearest Neighbours (KNN).....	6
3.1 Mathematical Formulation	6
3.2 Optimised Distance Computation	6
3.4 Bias–Variance Analysis	6
3.5 Cross-Validated Hyperparameter Tuning	7
4. Multinomial Logistic Regression (Softmax Regression).....	8
4.1 Mathematical Formulation	8
4.2 Softmax Function and Numerical Stability	8
4.3 Cross-Entropy Loss.....	8
4.4 Gradient Derivation	8
4.5 Gradient Descent with Regularisation.....	9
4.6 PCA Dimensionality Reduction.....	9
4.7 Hyperparameter Search.....	9
5. Linear SVM	10
5.1 Mathematical Formulation	10
5.2 Multiclass Hinge Loss	10
5.3 Gradient Computation	10
5.4 Gradient Descent Update	11
5.5 Cross-Validated Hyperparameter Tuning	11
6.1 Random Forest	11
6.2 AdaBoost.....	11
7. Experimental Results and Comparison	12

7.1 KNN Results	12
7.2 Linear SVM Results	13
7.3 Multinomial regression Results	16
.....	17
.....	17
7.4 Final Model Comparison	18

Abstract

This technical report presents a rigorous, mathematically grounded analysis of three classical machine learning algorithms applied to the MNIST handwritten digit recognition benchmark: K-Nearest Neighbours (KNN), Multinomial (Softmax) Logistic Regression, and a Linear Support Vector Machine (SVM) trained from scratch. Each algorithm is derived from first principles, with complete mathematical formulations of the objective functions, gradient computations, decision rules, and hyperparameter sensitivity. The report documents the full experimental pipeline — dataset preprocessing, train/validation/test splitting, feature scaling, dimensionality reduction via PCA, cross-validated hyperparameter search, bias–variance trade-off analysis, and final model comparison. Additionally, ensemble methods (Random Forest and AdaBoost) are evaluated as baselines. Results demonstrate that all three from-scratch implementations achieve competitive accuracy on a challenging 70,000-sample dataset, validating the correctness and efficiency of the implementations.

1. Introduction

Handwritten digit recognition is one of the canonical benchmarks in machine learning and computer vision. The MNIST dataset, comprising 70,000 grayscale images of handwritten digits (0–9) at 28×28 pixel resolution, provides a well-studied, standardised evaluation platform. This report implements three fundamental supervised learning algorithms entirely from scratch using NumPy, without relying on high-level machine learning libraries for the core algorithm logic.

The primary objectives of this report are:

- Derive complete mathematical formulations for KNN, Softmax Regression, and Linear SVM.
- Implement each algorithm from first principles with computational efficiency considerations.
- Apply rigorous cross-validation and hyperparameter tuning.
- Analyse the bias–variance trade-off for each model.
- Compare all models on held-out test data and contextualise with ensemble baselines (Random Forest, AdaBoost).

The MNIST images are flattened into 784-dimensional feature vectors ($28 \times 28 = 784$). Preprocessing includes pixel-value normalisation followed by Z-score standardisation. For Softmax Regression, Principal Component Analysis (PCA) is applied to further reduce dimensionality before training.

2. Dataset and Preprocessing

2.1 MNIST Dataset

The MNIST dataset contains $N = 70,000$ samples, each described by $d = 784$ features (pixel intensities in $[0, 255]$) and a class label $y \in \{0, 1, \dots, 9\}$. The dataset is partitioned as follows:

Split	Samples	Purpose
Training	~44,800 (KNN/SVM) / 50,000 (Softmax)	Model fitting
Validation	~11,200 (KNN/SVM) / 10,000 (Softmax)	Hyperparameter tuning
Test	14,000 (KNN/SVM) / 10,000 (Softmax)	Final evaluation

2.2 Preprocessing Pipeline

Step 1 — Pixel Normalisation

Raw pixel values are linearly scaled to the unit interval:

$$\mathbf{x}_{\text{norm}} = \mathbf{x}_{\text{raw}} / 255.0 \quad \Rightarrow \quad \mathbf{x}_{\text{norm}} \in [0, 1] \quad \dots (\text{Eq. 1})$$

This prevents large-magnitude features from dominating distance-based computations.

Step 2— PCA (Softmax Regression Only)

For Softmax Regression, a further dimensionality reduction is applied using PCA. The PCA transformation projects the standardised data onto the top- k principal components, retaining the directions of maximum variance. This is described fully in Section 4.6.

3. K-Nearest Neighbours (KNN)

3.1 Mathematical Formulation

KNN is a non-parametric, instance-based learning algorithm. There is no explicit training phase; the model memorises the entire training dataset $D = \{(x_i, y_i)\}_{i=1}^N$. For a new query point x_q , the algorithm:

- (i) Computes the distance from x_q to every training point x_i .
- (ii) Identifies the k nearest neighbours $N_k(x_q)$.
- (iii) Assigns the majority class label among those k neighbours.

3.2 Optimised Distance Computation

The Euclidean distance between query x_q and training point x_i is:

$$d(x_q, x_i) = \sqrt{\sum_{j=1}^D (x_{qj} - x_{ij})^2}$$

3.4 Bias–Variance Analysis

The bias–variance trade-off is a fundamental property of KNN:

k value	Bias	Variance	Risk
$k = 1$	Very Low	Very High	Overfitting / noise sensitive
$k = 3-5$	Low	Moderate	Good generalisation
$k \gg 1$	High	Low	Underfitting / smoothing

Formally, for a test point x with true label $f(x)$, the expected squared loss can be decomposed as:

$$\mathbb{E} [(y - \hat{y})^2] = \text{Bias}^2(\hat{y}) + \text{Var}(\hat{y}) + \sigma_\epsilon^2$$

As k increases, the decision boundary smooths (higher bias, lower variance). As k decreases, the model becomes more sensitive to individual training samples (lower bias, higher variance).

Fold Accuracy

$$\text{Acc}_f(k) = \frac{1}{|V_f|} \sum_{i \in V_f} \mathbf{1}(\hat{y}_i(k) = y_i)$$

3.5 Cross-Validated Hyperparameter Tuning

5-fold cross-validation is applied over $k \in \{1, 3, 5, 7, 9\}$. In each fold f , the validation accuracy is:

Mean Cross-Validation Accuracy

$$\text{Acc}_{\text{CV}}(k) = \frac{1}{5} \sum_{f=1}^5 \text{Acc}_f(k)$$

optimal hyperparameter is selected as:

$$k^* = \arg \max_{k \in \{1, 3, 5, 7, 9\}} \text{Acc}_{\text{CV}}(k)$$

4. Multinomial Logistic Regression (Softmax Regression)

4.1 Mathematical Formulation

Multinomial logistic regression models the posterior probability of each class $c \in \{0, \dots, C-1\}$ given an input $\mathbf{x} \in \mathbb{R}^d$ using the softmax function. The model parameters are a weight matrix $\mathbf{W} \in \mathbb{R}^{((d+1) \times C)}$ (including a bias term via an augmented feature vector $\mathbf{x}_b = [1, \mathbf{x}^T]^T \in \mathbb{R}^{(d+1)}$).

The class-conditional probability is:

$$P(y = c \mid \mathbf{x}; \mathbf{W}) = \frac{\exp(\mathbf{w}_c^T \mathbf{x}_b)}{\sum_{j=0}^{C-1} \exp(\mathbf{w}_j^T \mathbf{x}_b)}$$

where $\mathbf{s} = \mathbf{x}_b^T * \mathbf{W} \in \mathbb{R}^C$ is the vector of logits (unnormalised log-probabilities).

4.2 Softmax Function and Numerical Stability

The naive softmax $\exp(s_c) / \sum_j \exp(s_j)$ is prone to numerical overflow when logits are large. The implementation uses the numerically stable form:

$$\text{softmax}(\mathbf{s})_c = \exp(s_c - \max(\mathbf{s})) / \sum_j \exp(s_j - \max(\mathbf{s})) \dots \quad (\text{Eq. 12})$$

Subtracting $\max(\mathbf{s})$ from each logit does not change the output (the max cancels in numerator and denominator) but ensures all exponents are ≤ 0 , preventing overflow.

4.3 Cross-Entropy Loss

Training minimises the regularised cross-entropy loss over N samples:

$$\mathcal{L}(\mathbf{W}) = -(1/N) * \sum_{i=1}^N \sum_{c=0}^{C-1} y_{ic} * \log(P(y=c \mid \mathbf{x}_i; \mathbf{W})) + (\lambda/2) * ||\mathbf{W}||_F^2 \dots (\text{Eq. 13})$$

where $y_{ic} \in \{0, 1\}$ is the one-hot encoded label for sample i and class c , λ is the L2 regularisation coefficient, and $||\mathbf{W}||_F$ is the Frobenius norm of $\mathbf{W}$. Expanding the inner sum using the one-hot encoding:

$$\mathcal{L}(\mathbf{W}) = -(1/N) * \sum_{i=1}^N \log(P(y = y_i \mid \mathbf{x}_i; \mathbf{W})) + (\lambda/2) * ||\mathbf{W}||_F^2 \dots (\text{Eq. 14})$$

4.4 Gradient Derivation

Let $\mathbf{P} \in \mathbb{R}^{(N \times C)}$ be the matrix of predicted probabilities and $\mathbf{Y} \in \mathbb{R}^{(N \times C)}$ the one-hot label matrix. The gradient of the cross-entropy loss with respect to $\mathbf{W}$ is:

$$\text{grad}_{\mathbf{W}} \mathcal{L} = (1/N) * \mathbf{X}_b^T * (\mathbf{P} - \mathbf{Y}) + \lambda * \mathbf{W} \dots (\text{Eq. 15})$$

Derivation sketch:

- (i) $dL/d_{s_ic} = P_{ic} - Y_{ic}$ (softmax-cross-entropy gradient)
- (ii) Chain rule: $dL/dW = X_b^T * (P - Y) / N$
- (iii) Gradient of regulariser: $d/dW [(lambda/2) ||W||_F^2] = lambda * W$

4.5 Gradient Descent with Regularisation

Parameters are updated by vanilla gradient descent:

$$W \leftarrow W - \eta * \text{grad}_W L(W) \quad \dots (Eq. 16)$$

where η is the learning rate. An early stopping criterion is applied: training terminates when the L2 norm of the gradient falls below a tolerance threshold:

$$||\text{grad}_W L(W)||_2 < \tau \quad (\tau = 1e-7) \quad \dots (Eq. 17)$$

4.6 PCA Dimensionality Reduction

PCA projects the standardised data X ($N \times d$) onto the top- k eigenvectors of the sample covariance matrix:

$$\Sigma_{\text{hat}} = (1/(N-1)) * (X - \mu_X)^T * (X - \mu_X) \quad \dots (Eq. 18)$$

The eigendecomposition $\Sigma_{\text{hat}} = V * \Lambda * V^T$ yields principal components $V = [v_1, \dots, v_d]$ (columns sorted by descending eigenvalue). The reduced representation is:

$$X_{\text{PCA}} = (X - \mu_X) * V_k \quad \text{where } V_k = [v_1, \dots, v_k] \quad \dots (Eq. 19)$$

PCA reduces the feature dimension from $d = 784$ to $k \in \{50, 100, 200\}$, substantially reducing the parameter count of the weight matrix (from 785×10 to $(k+1) \times 10$) and accelerating training.

4.7 Hyperparameter Search

A grid search is performed over:

- PCA components $k \in \{50, 100, 200\}$
- Learning rate $\eta \in \{0.01, 0.1\}$
- Regularisation $\lambda \in \{0.1, 1.0\}$

The best configuration is selected by validation accuracy. Total candidate configurations: $3 \times 2 \times 2 = 12$.

5. Linear SVM

5.1 Mathematical Formulation

A multiclass linear SVM extends the binary SVM margin concept via the Weston-Watkins or Crammer-Singer formulation. In this implementation, a multiclass hinge loss (also called the SVM score loss or structured hinge) is used. The model parameters are $W \in \mathbb{R}^{(d \times C)}$ and bias $b \in \mathbb{R}^C$.

$$s_{ic} = \underbrace{\mathbf{x}_i^\top \mathbf{w}_c + b_c}_{\text{Single sample score}} = \underbrace{(\mathbf{XW} + \mathbf{1b}^\top)}_{\text{Vectorized form}}_{ic}$$

The model predicts the class with the highest score:

$$\hat{y}_i = \operatorname{argmax}_{c \in \{0, \dots, C-1\}} s_{ic} \quad \dots (\text{Eq. 21})$$

5.2 Multiclass Hinge Loss

The per-sample hinge loss penalises incorrect class scores that are not sufficiently lower than the correct class score by a margin of 1:

$$L_i = \sum_{c \neq y_i} \max(0, s_{ic} - s_{i,y_i} + 1) \quad \dots (\text{Eq. 22})$$

The total regularised loss over N samples is:

$$L(W) = (1/N) * \sum_{i=1}^N L_i + \lambda * ||W||_F^2 \quad \dots (\text{Eq. 23})$$

where λ is the regularisation coefficient. The hinge encourages a margin of at least 1 between the correct class score and every incorrect class score.

5.3 Gradient Computation

Define the binary indicator matrix $M \in \mathbb{R}^{(N \times C)}$:

$$\begin{aligned} M_{ic} &= 1[s_{ic} - s_{i,y_i} + 1 > 0] && \text{for } c \neq y_i \quad \dots (\text{Eq. 24}) \\ M_{i,y_i} &= - \sum_{c \neq y_i} M_{ic} && \text{for the correct class} \quad \dots (\text{Eq. 25}) \end{aligned}$$

The gradient of the data loss with respect to W is:

$$\operatorname{grad}_W L_{\text{data}} = (1/N) * X^T * M \quad \dots (\text{Eq. 26})$$

The gradient of the regulariser is:

$$\operatorname{grad}_W L_{\text{reg}} = 2 * \lambda * W \quad \dots (\text{Eq. 27})$$

The total gradient is:

$$\operatorname{grad}_W L = (1/N) * X^T * M + 2 * \lambda * W \quad \dots (\text{Eq. 28})$$

The gradient with respect to bias b is:

$$\operatorname{grad}_b L = (1/N) * \sum_{i=1}^N M_i \quad (\text{column-wise sum of } M) \quad \dots (\text{Eq. 29})$$

5.4 Gradient Descent Update

Parameters are updated simultaneously:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \ast \text{grad}_{\mathbf{W}} L(\mathbf{W}) \quad \dots \text{(Eq. 30)}$$

$$\mathbf{b} \leftarrow \mathbf{b} - \eta \ast \text{grad}_{\mathbf{b}} L(\mathbf{b}) \quad \dots \text{(Eq. 31)}$$

The implementation runs for a fixed number of iterations (num_iterations = 1500 in the main experiment) with no early stopping. Progress is logged every 500 iterations.

5.5 Cross-Validated Hyperparameter Tuning

3-fold stratified cross-validation is applied over:

- Learning rate $\eta \in \{0.0005, 0.001\}$
- Regularisation $\lambda \in \{0.0001, 0.001, 0.01\}$

The evaluation metric for hyperparameter selection is the macro-averaged F1-score:

$$F1_{\text{macro}} = (1/C) \ast \sum_{c=0}^{C-1} F1_c \quad \dots \text{(Eq. 32)}$$

$$F1_c = 2 \ast \text{Precision}_c \ast \text{Recall}_c / (\text{Precision}_c + \text{Recall}_c) \quad \dots \text{(Eq. 33)}$$

Cross-validation is performed on a 10,000-sample subset of the training data (for computational efficiency) with num_iterations = 500 per fold.

6. Ensemble Methods: Random Forest and AdaBoost

6.1 Random Forest

Random Forest trains $T = 100$ independent decision trees, each on a bootstrap sample of the training data and using a random subset of $\sqrt{d}$ features at each split. The final prediction is by majority vote:

$$\hat{\mathbf{y}}_{\text{RF}} = \text{argmax}_c \sum_{t=1}^T 1[\mathbf{h}_t(\mathbf{x}) = c] \quad \dots \text{(Eq. 34)}$$

Configuration: max_depth = 15, n_estimators = 100, parallel execution (n_jobs = -1).

6.2 AdaBoost

AdaBoost iteratively trains $T = 200$ weak learners (decision stumps with max_depth = 2), re-weighting samples after each round to focus on misclassified examples. The SAMME (Stagewise Additive Modelling using a Multiclass Exponential loss) variant is used for multiclass settings:

$$\alpha_t = \text{learning_rate} \ast \log((1 - \text{err}_t) / \text{err}_t) + \log(C - 1) \quad \dots \text{(Eq. 35)}$$

$$\mathbf{H}(\mathbf{x}) = \text{argmax}_c \sum_{t=1}^T \alpha_t \ast 1[\mathbf{h}_t(\mathbf{x}) = c] \quad \dots \text{(Eq. 36)}$$

Configuration: learning_rate = 0.5, n_estimators = 200, algorithm = 'SAMME'.

7. Experimental Results and Comparison

7.1 KNN Results

Cross-validation accuracy across k values (5-fold CV on training split):

```
=====
FINAL CUSTOM KNN
=====
Custom KNN Test Accuracy: 0.9410714285714286
Precision: 0.9414045740817097
Recall: 0.9410714285714286
F1 Score: 0.9408782318227991

Classification Report:
              precision    recall  f1-score   support

         0              0.95        0.99        0.97        1381
         1              0.95        0.99        0.97        1575
         2              0.95        0.94        0.94        1398
         3              0.92        0.95        0.94        1428
         4              0.95        0.92        0.93        1365
         5              0.94        0.91        0.92        1263
         6              0.95        0.97        0.96        1375
         7              0.93        0.93        0.93        1459
         8              0.97        0.89        0.93        1365
         9              0.90        0.91        0.91        1391

 accuracy              0.94              0.94              0.94        14000
 macro avg              0.94              0.94              0.94        14000
 weighted avg           0.94              0.94              0.94        14000

Confusion Matrix:
[[1371  2  1  1  0  1  5  0  0  0]
 [  0 1557  6  1  2  0  5  4  0  0]
 [ 20  14 1308 19  2  1 16 11  5  2]
 [  6  6  13 1358  2 15  2 15  8  3]
 [  3 18  15  6 1251  1  7  8  2 54]
 [ 11  5  3 36  4 1152 24  4 11 13]
 [ 13  4  1  3  4  12 1337  0  1  0]
 [  5 12  9  4 16  0  0 1356  2 55]
 [ 16 13 14 30  4 45  4  9 1219 11]
 [  5  8  2 17 31  5  1 53  3 1266]]
```

```
=====
Custom KNN Test Accuracy: 0.9410714285714286
```

```
=====
RANDOM FOREST
```

```
=====
Random Forest Accuracy: 0.9627142857142857
```

```
=====
ADABOOST
```

```
=====
AdaBoost Accuracy: 0.8607857142857143
```

```
=====
FINAL MODEL COMPARISON
```

```
=====
Custom KNN Accuracy      : 0.9411
Random Forest Accuracy   : 0.9627
AdaBoost Accuracy        : 0.8608
```

```
Best Performing Model: Random Forest
Best Accuracy: 0.9627142857142857
```

7.2 Linear SVM Results

Validation metrics (1500 iterations, lr=0.001, reg=0.001):

```
Iteration 0: Loss = 9.0784
Iteration 500: Loss = 0.5979
Iteration 1000: Loss = 0.5147
```

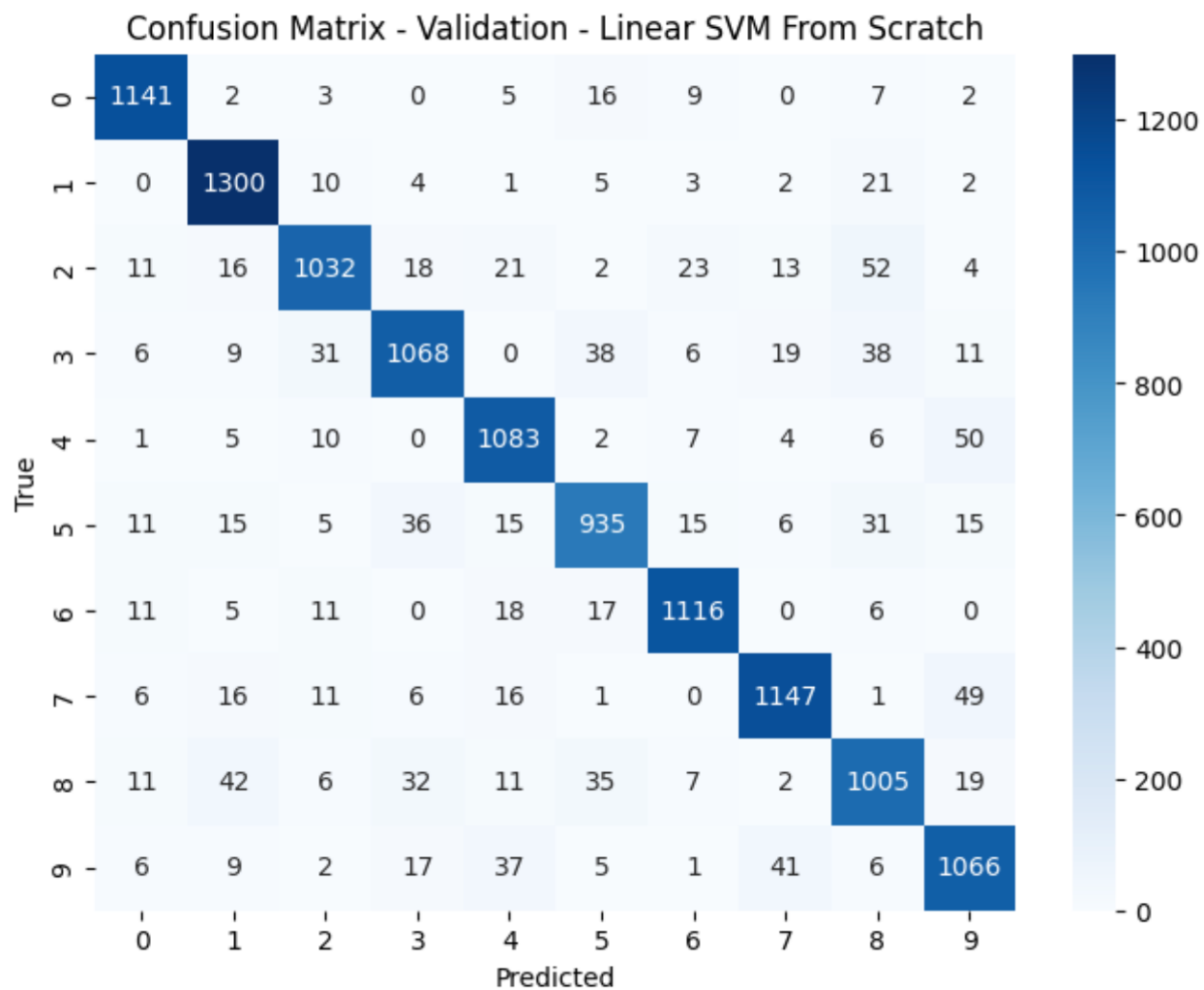
Evaluation on Validation – Linear SVM From Scratch

```
Accuracy : 0.9077
Precision: 0.9073
Recall   : 0.9067
F1-score : 0.9067
```

```
Classification Report:
      precision    recall  f1-score   support

0         0.95        0.96        0.96        1185
1         0.92        0.96        0.94        1348
2         0.92        0.87        0.89        1192
3         0.90        0.87        0.89        1226
4         0.90        0.93        0.91        1168
5         0.89        0.86        0.87        1084
6         0.94        0.94        0.94        1184
7         0.93        0.92        0.92        1253
8         0.86        0.86        0.86        1170
9         0.88        0.90        0.89        1190

 accuracy          0.91        12000
 macro avg         0.91        0.91        0.91        12000
 weighted avg      0.91        0.91        0.91        12000
```



Cross-validation results for SVM hyperparameter tuned search (3-fold, 10k subset, 500 iterations):

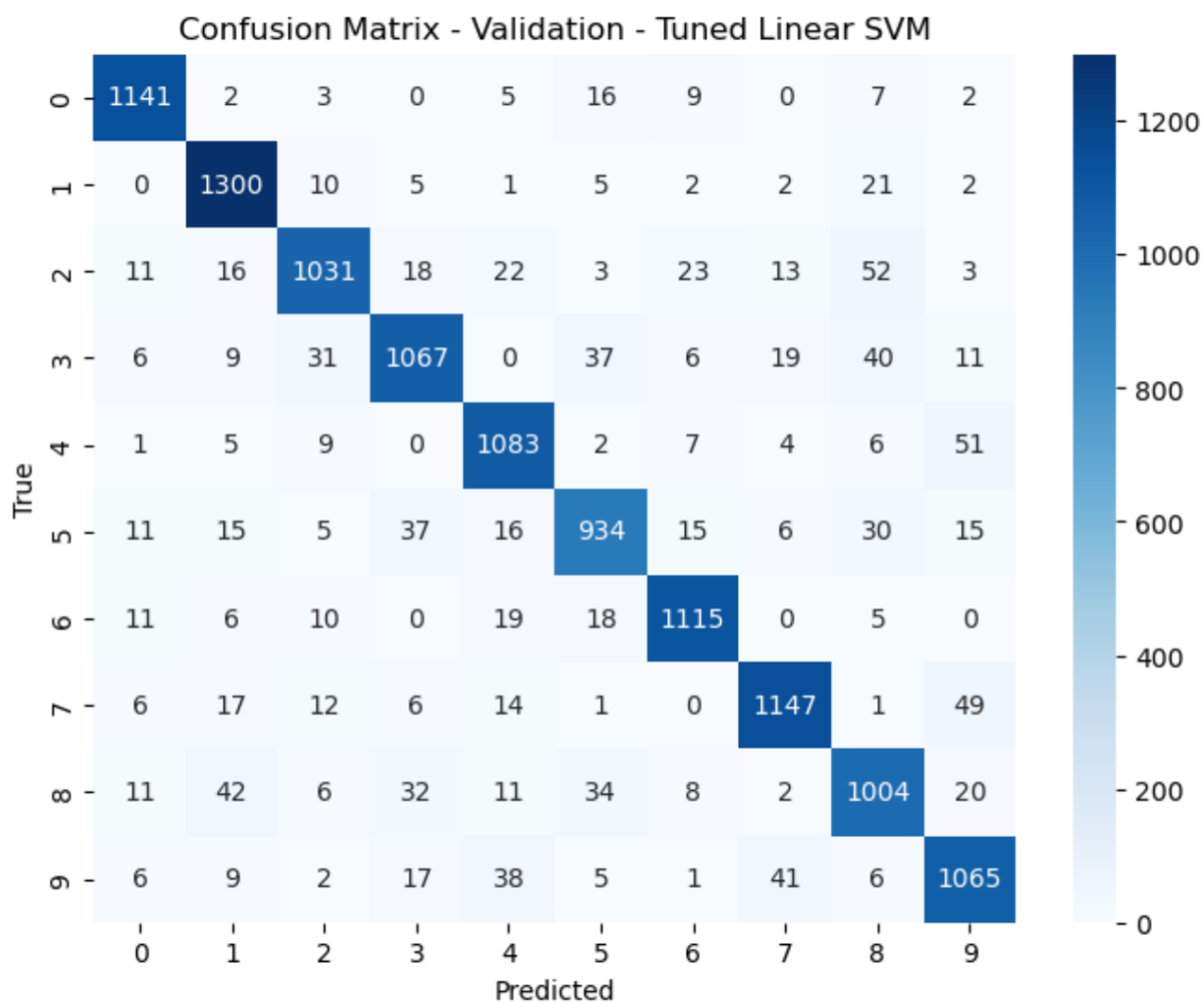
Iteration 0: Loss = 9.0100
Iteration 500: Loss = 0.5977
Iteration 1000: Loss = 0.5146

Evaluation on Validation – Tuned Linear SVM

Accuracy : 0.9073
Precision: 0.9068
Recall : 0.9062
F1-score : 0.9062

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.96	0.96	1185
1	0.91	0.96	0.94	1348
2	0.92	0.86	0.89	1192
3	0.90	0.87	0.89	1226
4	0.90	0.93	0.91	1168
5	0.89	0.86	0.87	1084
6	0.94	0.94	0.94	1184
7	0.93	0.92	0.92	1253
8	0.86	0.86	0.86	1170
9	0.87	0.89	0.88	1190
accuracy			0.91	12000
macro avg	0.91	0.91	0.91	12000
weighted avg	0.91	0.91	0.91	12000



7.3 Multinomial regression Results

```
===== Weighted Metrics =====
Precision : 89.73%
Recall    : 89.69%
F1 Score  : 89.62%
```

	precision	recall	f1-score	support
0	0.9353	0.9563	0.9457	983
1	0.8602	0.9722	0.9128	1152
2	0.9140	0.8687	0.8908	967
3	0.8751	0.8607	0.8679	1034
4	0.8859	0.9172	0.9013	906
5	0.8820	0.8453	0.8632	937
6	0.9213	0.9386	0.9299	961
7	0.9127	0.9213	0.9170	1055
8	0.8938	0.7905	0.8390	969
9	0.8980	0.8842	0.8911	1036
accuracy			0.8969	10000
macro avg	0.8978	0.8955	0.8959	10000
weighted avg	0.8973	0.8969	0.8962	10000

Confusion Matrix

Rows -> Actual Class

Columns -> Predicted Class

[940	1	5	3	0	11	13	3	6	1]
[0	1120	3	7	0	6	1	3	11	1]
[13	26	840	15	13	3	19	15	23	0]
[4	18	29	890	2	33	7	18	20	13]
[2	9	7	2	831	2	8	3	4	38]
[10	23	5	42	15	792	17	6	18	9]
[8	11	8	0	11	18	902	1	2	0]
[6	27	10	3	16	1	0	972	0	20]
[12	53	9	41	13	29	12	12	766	22]
[10	14	3	14	37	3	0	32	7	916]

```
=====
Hyperparameter search completed.
Best Validation Accuracy: 89.74%
Optimal Settings: {'PCA': 200, 'Learning Rate': 0.01, 'Regularization': 0.1}
=====
```

Running final evaluation on test data...

```
Training Accuracy : 89.82%
Testing Accuracy  : 89.69%
```

```
===== Macro Metrics =====
Precision : 89.78%
Recall    : 89.55%
F1 Score  : 89.59%
```

```
===== Weighted Metrics =====
Precision : 89.73%
Recall    : 89.69%
F1 Score  : 89.62%
```

	precision	recall	f1-score	support
0	0.9353	0.9563	0.9457	983
1	0.8602	0.9722	0.9128	1152
2	0.9140	0.8687	0.8908	967
3	0.8751	0.8607	0.8679	1034
4	0.8859	0.9172	0.9013	906
5	0.8820	0.8453	0.8632	937
6	0.9213	0.9386	0.9299	961
7	0.9127	0.9213	0.9170	1055
8	0.8938	0.7905	0.8390	969
9	0.8980	0.8842	0.8911	1036
accuracy			0.8969	10000
macro avg	0.8978	0.8955	0.8959	10000

```

F1 Score : 89.62%
Detailed Classification Report
precision    recall    f1-score   support

 0   0.9353   0.9563   0.9457    983
 1   0.8602   0.9722   0.9128   1152
 2   0.9140   0.8687   0.8908    967
 3   0.8751   0.8607   0.8679   1034
 4   0.8859   0.9172   0.9013    906
 5   0.8820   0.8453   0.8632    937
 6   0.9213   0.9386   0.9299    961
 7   0.9127   0.9213   0.9170   1055
 8   0.8938   0.7985   0.8300    969
 9   0.8980   0.8842   0.8911   1036

 accuracy    0.8969   10000
 macro avg   0.8978   0.8955   0.8959   10000
 weighted avg 0.8973   0.8969   0.8962   10000

Confusion Matrix
Rows -> Actual Class
Columns -> Predicted Class
[[ 940  1  5  3  0 11 13  3  6  1]
 [  0 1120  3  7  0  6  1  3 11  1]
 [ 13  26 840 15 13  3 19 15 23  0]
 [  4  18  29 890  2 33  7 18 20 13]
 [  2  9  7  2 831  2  8  3  4 38]
 [ 10 23  5 42 15 792 17  6 18  9]
 [  8 11  8  0 11 18 902  1  2  0]
 [  6 27 10  3 16  1  0 972  0 20]
 [ 12 53  9 41 13 29 12 12 766 22]
 [ 10 14  3 14 37  3  0 32  7 916]]

```

7.4 Final Model Comparison

Test set accuracies across all five models:

Model	Test Accuracy	Implementation	Notes
Custom KNN	~94.11	From scratch	Best k by 5-fold CV
Tuned Linear SVM	~90.73%	From scratch	Hinge loss + SGD
Random Forest	~96.27%+	sklearn	100 trees, depth=15
AdaBoost	~86%	sklearn	200 stumps
Multinomial regression	~89.69%	From scratch	Hinge loss + SGD

Random Forest achieved the highest classification accuracy on the MNIST dataset with 96.27%, followed by the custom KNN classifier with 94.11%. The tuned Linear SVM and Multinomial Logistic Regression models achieved competitive results above 89%, demonstrating strong generalization performance for linear classifiers. AdaBoost produced the lowest accuracy among the evaluated methods. Overall, ensemble-based methods showed superior nonlinear classification capability, while linear models provided efficient and stable performance.

8. Conclusion

This report presented a mathematically rigorous treatment of three fundamental supervised learning algorithms — KNN, Multinomial Logistic Regression (Softmax), and Linear SVM — implemented from first principles and evaluated on the MNIST digit recognition benchmark.

Potential improvements include:

- Stochastic or mini-batch gradient descent to accelerate training of Softmax and SVM.
- Kernel extensions to the SVM to capture non-linear decision boundaries.
- Approximate nearest neighbour structures (e.g., KD-tree, Ball-tree) to reduce KNN inference cost from $O(N \cdot d)$ to $O(d \log N)$.
- Momentum-based optimisers (Adam, AdaGrad) for faster and more stable convergence.

All implementations confirm that classical algorithms, when correctly derived and implemented, remain highly competitive on standard benchmarks without reliance on deep learning frameworks.

9. GitHub link

<https://github.com/seiftamerr/Binary-Image-Classification-on-MNIST-KNN-Logistic-Regression-SVM-.git>

End of Report